



WPYEG • Edmonton WordPress Meetup

Better by Default

Sane defaults for every new WordPress site — one toggle at a time.

A hands-on workshop · the better-by-default plugin

A fresh WordPress install ships wide open

Out of the box, core leaves a lot of doors ajar and a lot of noise switched on. None of it is a bug — it's just defaults chosen for maximum compatibility, not for your site.



Username leak

REST + author archives happily list every login name to anonymous visitors.



Legacy XML-RPC wide open

pingback and system.multicall amplifiers answer by default.



Dead weight loads

Emoji scripts, version tags, and RSD links ship on every single page view.



Spam surface invites

Comments, pingbacks, and trackbacks are open by default on new content.

The one idea to take home

A “default” is just an opinionated filter behind a toggle.

```
if ( wpyeg_defaults_enabled( 'restrict_rest_user_discovery' ) ) {  
    add_filter( 'rest_endpoints', $hide_users_endpoint );  
} // that's the whole pattern, repeated ~20 times
```

Two words, gently: hooks & filters

WordPress is built to be interrupted on purpose. You don't edit core — you hop in at labeled moments.

Action

A

“When you reach this moment, also DO this.” A doorbell you answer.

```
add_action( 'init', fn );
```

Filter

F

“Before you use this value, let me CHANGE it first.” A mail slot that edits the letter.

```
add_filter( 'xmlrpc_enabled',  
    '__return_false' );
```

Six categories of default

1

Security

Shrink the attack surface

2

Content

Close public spam & leaks

3

Admin UX

Calmer, faster dashboard

4

Login

Sessions & credentials

5

Branding

Own the login screen

6

Performance

Trim the page weight

1

SECTION ONE

Security & Attack Surface

Every item here removes something an attacker can poke — usually in one line.

Restrict REST API user discovery

The `/wp/v2/users` endpoint hands out every author's login name to anyone — half of a brute-force guess, free. We keep REST working for logged-in tools but slam the door on anonymous enumeration.

OPTION `wpyeg_restrict_rest_user_discovery`
DEFAULT
`yes`

```
add_filter( 'rest_endpoints', function ( $ep ) {  
    if ( ! is_user_logged_in() ) {  
        unset( $ep['/wp/v2/users'] );  
        unset(  
            $ep['/wp/v2/users/(?P<id>[\d]+)']  
        );  
    }  
    return $ep;  
} );
```

Lock XML-RPC down by category

XML-RPC is an old switchboard — every method is a line. We unplug the lines we don't want: pingbacks (spam/DDoS) and the credential-authenticated blogging APIs (the brute-force target), via a method filter. `system.multicall` can't be filtered off, so a replacement server refuses it. Jetpack's `jetpack.*` lines stay untouched.

```
OPTION    wpyeg_xmlrpc_allow_pingbacks /  
_remote_publishing / _multicall  
DEFAULT  
no (each)
```

```
// each category off → remove its methods  
add_filter( 'xmlrpc_methods', function ( $m ) {  
    if ( ! allow( 'pingbacks' ) )  
        unset( $m['pingback.ping'] );  
    if ( ! allow( 'remote_publishing' ) )  
        // drop wp.* metaWeblog.* mt.* blogger.*  
        return $m;  
} );  
// multicall: swap in a server that refuses it  
add_filter( 'wp_xmlrpc_server_class', $refuse );
```

Keep Application Passwords available

The one we don't lock down. An Application Password is a spare key cut for one app — hashed, per-application, revocable on its own. It's the safer REST credential, and the only one core accepts for REST Basic Auth. So they stay on; prohibit them only if policy forbids non-interactive credentials.

```
OPTION    wpypg_disable_application_passwords
DEFAULT
no (available)
```

```
// available by default -
// prohibit only if opted in
if ( wpypg_defaults_enabled(
    'disable_application_passwords' ) ) {
    add_filter(
        'wp_is_application_passwords_available',
        '__return_false'
    );
}
```

Require strong passwords

The rules changed: NIST 800-63B and OWASP now say length plus breach screening beats composition rules. Forcing upper/lower/number/symbol just herds everyone to Password1! — predictable, not strong. We require 15+ chars and screen against known breaches, server-side.

```
OPTION    wpyeg_require_strong_passwords
DEFAULT
yes
```

```
// hooked on user_profile_update_errors
if ( strlen( $pw ) < 15 ) {
    $errors->add( 'short', 'Use 15+ chars.' );
}

// screen against breaches (HIBP),
// not composition rules
if ( wpyeg_password_is_pwned( $pw ) ) {
    $errors->add( 'pwned', 'Seen in a breach.' );
}
```

Remove fingerprints, add headers

Two quick wins: stop broadcasting your exact core version to vulnerability scanners, and send three low-risk security headers that most sites can adopt without breaking anything.

```
OPTION    wpyeg_remove_version / _security_headers
DEFAULT
yes / yes
```

```
remove_action( 'wp_head', 'wp_generator' );

add_filter( 'wp_headers', function ( $h ) {
    $h['X-Content-Type-Options'] = 'nosniff';
    $h['X-Frame-Options']       = 'SAMEORIGIN';
    $h['Referrer-Policy'] =
        'strict-origin-when-cross-origin';
    return $h;
} );
```

Lock REST to logged-in users

The nuclear option. Requiring auth for ALL REST calls stops anonymous scraping cold — but it also breaks the block editor and many blocks. That's why it ships OFF. Reach for it only on pure brochure sites.

OPTION `wpyeg_disable_rest`
DEFAULT
`no`

```
add_filter( 'rest_authentication_errors',  
function ( $result ) {  
    if ( ! empty( $result ) ) return $result;  
    if ( ! is_user_logged_in() ) {  
        return new WP_Error(  
            'rest_not_logged_in', 'Auth required.',  
            array( 'status' => 401 ) );  
    }  
    return $result;  
} );
```



2

SECTION TWO

Content & Public Surfaces

Close the spam funnels and the thin pages Google (and bots) love to crawl.

Disable comments, trackbacks & pingbacks

For most business sites, comments are a spam magnet with little upside. We close them everywhere, hide existing threads, drop the admin menu, and default new posts to pings-closed.

```
OPTION    wpjeg_disable_comments
DEFAULT
yes
```

```
add_filter( 'comments_open', '__return_false', 20 );
add_filter( 'pings_open',    '__return_false', 20 );
add_filter( 'comments_array',
            '__return_empty_array', 20 );

// + remove_post_type_support() on init
// + remove_menu_page('edit-comments.php')
// + drop the admin-bar comments node
```

Redirect author & attachment pages

Two thin-content leaks, same fix. Author archives expose login slugs; attachment pages are near-empty media wrappers. Both dilute SEO and add surface. Send them home with a 301.

```
OPTION    wpyeg_disable_author_archives /  
_redirect_attachment_pages  
DEFAULT  
yes / yes
```

```
add_action( 'template_redirect', function () {  
    if ( is_author() ) {  
        wp_safe_redirect( home_url('/'), 301 );  
        exit;  
    }  
    if ( is_attachment() ) {  
        // 301 to parent post, or home  
    }  
} );
```

Disable the emoji script

Core injects an emoji-detection script and inline CSS on every page load, plus a DNS-prefetch hint. Modern browsers render emoji natively, so this is pure dead weight you can shed.

OPTION `wpyeg_disable_emojis`
DEFAULT
`yes`

```
add_action( 'init', function () {  
    remove_action( 'wp_head',  
        'print_emoji_detection_script', 7 );  
    remove_action( 'wp_print_styles',  
        'print_emoji_styles' );  
    // ...admin + feed + mail variants too  
    add_filter( 'emoji_svg_url',  
        '__return_false' );  
} );
```



3

SECTION THREE

Admin UX & Login Sessions

Small quality-of-life defaults: a calmer dashboard and sensible session policy.

Faster search, quieter admin bar

On big sites the admin list-table search reads every word of every post — and crawls. Title-only search checks just the spines, far faster (off by default — it changes editor expectations). And the floating front-end admin bar can be hidden for non-admins.

```
OPTION    wpyeg_title_only_admin_search /  
_frontend_admin_bar_behavior  
DEFAULT  
no / ''
```

```
// title-only admin search — narrow the COLUMNS  
add_filter( 'post_search_columns',  
    function ( $cols, $s, $q ) {  
        if ( is_admin() && $q->is_main_query() )  
            return array( 'post_title' );  
        return $cols;    // front-end untouched  
    }, 10, 3 );  
// hide bar for non-admins  
add_filter( 'show_admin_bar', fn( $s ) =>  
    current_user_can('manage_options') ? $s : false );
```

Right-size the login session

Core's “Remember Me” lasts 14 days — often too long. Cap it (say 5 days), optionally shorten regular logins, or hide the checkbox entirely for shared machines. One filter does all three.

```
OPTION    wpyeg_remember_me_days /  
_session_regular_hours  
DEFAULT  
5 / 0
```

```
add_filter( 'auth_cookie_expiration',  
function ( $exp, $uid, $remember ) {  
    if ( $remember ) {  
        return 5 * DAY_IN_SECONDS;  
    }  
    return 12 * HOUR_IN_SECONDS;  
}, 10, 3 );  
  
// DAY_IN_SECONDS: core time constant
```



4

SECTION FOUR

Branding & Performance

Own the login screen, then shave the last bit of weight off every page.

Own the login screen

The login page is the site's front door — and the default “W” on wp-login.php links out to wordpress.org, a subtle trust leak. But repainting a client's front door uninvited is rude, so the default is to leave it alone; removing, unlinking, or replacing the logo is an opt-in, and any change points the link home.

```
OPTION    wpyeg_login_logo_behavior
DEFAULT
keep_default (keep / remove / unlink / replace)
```

```
// remove, unlink, or replace — a choice
add_action( 'login_head', $logo_css );

// any change points the link home
// (no separate toggle)
add_filter( 'login_headerurl', 'home_url' );
add_filter( 'login_headertext', fn() =>
    get_bloginfo( 'name' ) );
```

Throttle Heartbeat, defer scripts

Two opt-in levers. The Heartbeat API polls admin-ajax every 15–60s — throttle it (and drop it on the dashboard home) to ease shared hosting. And defer non-critical front-end scripts so they stop blocking render.

OPTION `wpyeg_throttle_heartbeat / _defer_scripts`
DEFAULT
`no / no`

```
add_filter( 'heartbeat_settings', fn( $s ) => {  
    $s['interval'] = 60; return $s;  
} );  
  
add_filter( 'script_loader_tag',  
    function ( $tag, $handle ) {  
        // add 'defer' (skip jquery-core)  
        return $tag;  
    }, 10, 2 );
```

Three things a plugin can't toggle

Some defaults live in wp-config.php, above the plugin layer. Document these as manual steps in your onboarding checklist.

```
define( 'DISALLOW_FILE_EDIT', true ); // no in-dashboard code editor
define( 'AUTOSAVE_INTERVAL', 120 ); // gentler autosave (seconds)
define( 'WP_POST_REVISIONS', 10 ); // cap revision-table bloat
```

1

Kills the theme/plugin editor

A stolen admin login can't rewrite your PHP from the dashboard.

2

Writes to the DB less often

Fewer autosave revisions during long edits.

3

Keeps revisions in check

Ten per post instead of unbounded growth.

How the plugin is built

One array is the whole map. Read it and you understand the plugin.

1 `schema()`

One array: every setting, its default, type & group. The single source of truth.



2 `settings page`

Loops the schema to render toggles under Settings → Better by Default.



3 `bootstrap()`

For each ENABLED key, wires its `add_filter` / `add_action` to the right hook.

```
$stored = get_option('wpyeg_better_by_default'); // read once
foreach ( wpyeg_defaults_schema() as $key => $field ) { /* render + wire */ }
```

Hands-on: install & flip switches

1 Upload the plugin

Plugins → Add New → Upload Plugin → choose better-by-default.zip → Activate.

2 Open the settings

Settings → Better by Default. Every toggle is grouped by category.

3 Verify a default

Visit `/wp-json/wp/v2/users` logged out — 401 or empty, not a list of usernames.

4 Toggle & re-check

Flip a switch off, reload, watch the behavior change. That's the filter turning on and off.

prefer the terminal? `wp plugin install ./better-by-default.zip --activate`

Your turn

Add one new default to the plugin

Goal: disable the WordPress dashboard “Welcome” panel. Two small edits — no new settings-page code.

```
// 1) add a schema entry in wpyeg_defaults_schema()
'hide_welcome_panel' => array(
    'default' => 'yes', 'type' => 'toggle', 'group' => 'ux',
    'label' => 'Hide dashboard welcome panel',
),

// 2) wire it inside wpyeg_defaults_bootstrap()
if ( wpyeg_defaults_enabled( 'hide_welcome_panel' ) ) {
    remove_action( 'welcome_panel', 'wp_welcome_panel' );
}
```

The cheat sheet

Defaults that ship ON — safe for nearly any site.

Default	Core hook	Category
Restrict REST user discovery	<code>rest_endpoints</code>	Security
Lock down XML-RPC by category	<code>xmlrpc_methods</code> / <code>wp_xmlrpc_server_class</code>	Security
Require strong passwords (15+ / breach-screened)	<code>user_profile_update_errors</code>	Security
Remove version + security headers	<code>wp_generator</code> / <code>wp_headers</code>	Security
Disable comments & pingbacks	<code>comments_open</code> / <code>pings_open</code>	Content
Redirect author + attachment pages	<code>template_redirect</code>	Content / SEO
Disable emoji script	<code>init</code> (remove_action)	Performance



Thanks, WPYEG!

Take the plugin, fork it, teach it. A default is just an opinionated filter behind a toggle.

Files: `better-by-default.zip` · `wordpress-default-settings.md`

Questions? License GPL-2.0-or-later — ship it.